WPT Day 5

Sun 25 Oct 26 · Callback nesting & callback hell, Promises (states / `.then` / `.catch`), `async` / `await`, DOM tree & DOM manipulation, HTML events, and form validation with live error messages — explained from first principles

---

Table of Contents

1. Callbacks That Must Wait: The Async Coordination Problem
2. Callback Nesting for Dependent Operations
3. Callback Hell
4. JavaScript Promises: Concept and States
5. Creating and Resolving Promises
6. Handling Promises: `.then()` and `.catch()`
7. Async / Await
8. DOM: Document Object Model
9. DOM Manipulation: Core Operations
10. Handling Events in HTML
11. Form Input, Validation, and Live Error Messages
12. Combined Worked Example: Promise Pipeline + Live Form
13. Quick Revision Checklist
14. Practice Exercises
15. Practice Answers

Callbacks That Must Wait: The Async Coordination Problem

Problem: the next line does not wait

Day 4 introduced callbacks and `setTimeout`: a callback is a function you hand to another function so *that* function can call it later. Day 5 starts from the practical consequence that students keep underestimating:

Callbacks never execute immediately. They wait for a triggering event — timer completion, a server response, a database read or write finishing, and so on. The lines of code *after* you schedule the async work do **not** pause. They run right away, usually before the callback has run at all.

Background: JavaScript in the browser (and Node.js) is single-threaded with an event loop. Async APIs register work and return control immediately; when the work finishes, the runtime queues your callback. That is why "call order" and "result order" can differ — and why code that *looks* sequential on the page is not sequential in time.

Library functions stay async

Async functions that come from libraries (Node.js, React, Angular, browser APIs, database drivers) **cannot have their execution nature changed** by the application developer.

- If a library function is async, **it stays async**.
- You must **adapt your logic around it** — put dependent work inside the callback (or later, inside `.then` / `await`), not on the next line.

You do not get to say "please make `readDataFromDbAsync` return the rows synchronously." The API is fixed; your sequencing is what you redesign.

Lecture's problem case: read DB, then send the same data

Imagine two helpers:

- `readDataFromDbAsync(callback)` — starts a DB read; later calls `callback(data)`.
- `sendResponse(data)` — sends an HTTP response with that data.

The broken pattern looks sequential but is not:

```
// BROKEN — sendResponse does not wait for the DB
let data = readDataFromDbAsync(function (rows) {
    // this runs later, when the DB finishes
});
sendResponse(data);  // runs NOW — data is still undefined / incomplete
```

What actually happens:

1. `readDataFromDbAsync` starts the read and returns immediately.
2. `sendResponse` runs **before** the DB finishes.
3. By the time the DB result arrives inside the callback, **the response has already been sent**.

Fix: put the dependent step inside the callback

```
// FIXED — send only after the read completes
readDataFromDbAsync(function (rows) {
    sendResponse(rows);
});
```

Rule in one sentence: **whatever needs the async result must live inside the callback that receives that result** (or in a Promise/`await` chain that is equivalent).

> **Exam:** Callbacks wait for a triggering event; they do not run on the next line. Async library functions stay async — you nest (or chain) dependent work inside their completion handlers.

> **Pitfall:** Calling `sendResponse` on the line after `readDataFromDbAsync` and expecting the DB data to be ready. It will not be.

Callback Nesting for Dependent Operations

When nesting is required

When **multiple async functions must run sequentially** because each step's **output is the next step's input**, each subsequent call goes **inside** the previous callback:

1. Read completes → write starts (inside read's callback).
2. Write completes → send response (inside write's callback).

If the operations are **independent** (none needs another's result), running them **simultaneously** is correct; nesting them would only waste wall-clock time. Nesting is required **only** when step *N* feeds step *N+1*.

Situation	Pattern
Step B needs Step A's result	Nest B inside A's callback (or chain Promises / `await`)
Steps are independent	Schedule them in parallel; do not nest
Mix of both	Parallel where possible; nest/chain only the dependent edges

Lecture demo: sum → factorial → square with `setTimeout`

The lecture simulates three **dependent** operations with timers (not real math libraries — the delays stand in for network/DB latency):

Step	Simulated work	Timer
Sum	`1.5s`	first
Factorial	`3s`	starts only after sum's callback
Square	`2s`	starts only after factorial's callback

Because of nesting, the timers are **sequential, not parallel**. Total wait ≈ **6.5 seconds** (1.5 + 3 + 2), not "whichever finishes last among three overlapping timers."

```
// Dependent chain: each step starts only when the previous callback fires
console.log("sum started");
setTimeout(function () {
    let sumResult = 10 + 20;
    console.log("sum result", sumResult);

    console.log("factorial started");
    setTimeout(function () {
        let n = 5, f = 1;
        for (let i = 1; i ≤ n; i++) {
            f *= i;
        }
        console.log("factorial result", f);

        console.log("square started");
        setTimeout(function () {
            let squareResult = 12 * 12;
            console.log("square result", squareResult);
            console.log("all dependent steps done");
        }, 2000);   // square: 2s, after factorial

    }, 3000);       // factorial: 3s, after sum

}, 1500);           // sum: 1.5s
```

Contrast with Day 4's **independent** timers: three `setTimeout` calls at the top level finish in delay order (sum → square → factorial), and the total wall time is about **3s** (the longest timer), not 6.5s.

> **Exam:** Nest only when output of step N is input of step N+1. Nested dependent timers add; independent timers overlap.

> **Pitfall:** Nesting independent work "for neatness." That artificially serialises work that could have finished sooner in parallel.

Callback Hell

The pyramid problem

Deep nesting of callbacks creates a **"pyramid" / triangle** code shape — each dependent step indents further to the right. The lecture names this **callback hell** because **the code structure itself becomes the obstacle**.

Typical symptoms of the shape:

```
asyncA(function (a) {
    asyncB(a, function (b) {
        asyncC(b, function (c) {
            asyncD(c, function (d) {
                asyncE(d, function (e) {
                    // logic buried here
                });
            });
        });
    });
});
```

Problems introduced by callback hell

Problem	Why it hurts
Poor readability	Eye must track deep indentation and matching braces
Hard to maintain and debug	Error paths and "which step failed?" are unclear
Hard to scale as features grow	One more dependent step means one more nesting level

Solution direction

Avoid nesting; use Promises instead (and later `async` / `await` on top of Promises). The rest of the morning is that escape hatch — same async reality, flatter control flow.

> **Exam:** Callback hell = deep nested callbacks → pyramid shape → poor readability, maintainability, debuggability, scalability. Cure: Promises / async-await, not "more nesting discipline."

JavaScript Promises: Concept and States

Real-life analogy

A Promise is inspired by a **real-life promise**: you commit to a future outcome, but the outcome is **unknown until the operation completes**. Until then, the promise is outstanding; afterwards it is either kept or broken.

Three states

State	Meaning
Pending	Promise created; operation not yet complete
Fulfilled / Resolved	Operation completed successfully
Rejected	Operation failed (server error, code error, invalid input, etc.)

Important ownership rule from the lecture:

- The **developer is solely responsible** for deciding when to `resolve` or `reject`.
- If **neither** is called, the promise stays in **pending forever** — a silent hang that is easy to miss in demos.

Executor runs immediately

The **Promise executor** (the function you pass to `new Promise( ... )`) **runs immediately** when the promise object is created. No button click or user input is required. If the `new Promise` sits at script top level, the executor runs **at page load**.

Background: Creating a Promise does not by itself wait for anything. The executor typically *starts* async work (or runs a sync check) and later calls `resolve` / `reject`. The Promise object is the handle others use to attach `.then` / `.catch`.

> **Exam:** Pending → Fulfilled or Rejected. Never calling resolve/reject leaves the promise pending forever. The executor runs as soon as `new Promise` runs.

Creating and Resolving Promises

Syntax

```
const p = new Promise(function (resolve, reject) {
    // your logic here
    // call resolve(data) on success
    // call reject(errorMsg) on failure
});
```

Facts the lecture stresses:

- The executor callback receives **two auto-populated function arguments**: conventionally named `resolve` and `reject`.
- **`resolve` and `reject` are not keywords.** Any parameter names work (the lecture jokes with names like `baboo`, `shona`) — what matters is calling the success function vs the failure function.
- Data passed to `resolve` is later received by `.then()`; data passed to `reject` is received by `.catch()`.

Example: even-number promise

```
function checkEven(n) {
    return new Promise(function (resolve, reject) {
        if (n % 2 === 0) {
            resolve("number is even");
        } else {
            reject("number is not even");
        }
    });
}

checkEven(4).then(function (msg) {
    console.log("OK:", msg);
}).catch(function (err) {
    console.log("Fail:", err);
});

checkEven(3).then(function (msg) {
    console.log("OK:", msg);
}).catch(function (err) {
    console.log("Fail:", err);
});
```

Functions that return promises do not return “direct values”

If a function **returns a Promise**, callers must **not** expect a normal return value on the next line:

```
// WRONG expectation
let result = checkEven(4);   // result is a Promise object, not the string
console.log(result);         // Promise { <pending> } or similar — not "number is
even"
```

You must handle via `.then()` / `.catch()` (or `await`) to get the actual resolved/rejected payload.

Wrapping async-style operations: sum, factorial, square

The lecture wraps the earlier math pipeline as **functions that return promises** — resolve with the numeric result, reject on invalid input (example rule: reject sum if either value exceeds 100):

```
function sum(a, b) {
    return new Promise(function (resolve, reject) {
        setTimeout(function () {
            if (a > 100 || b > 100) {
                reject("values must not exceed 100");
                return;
            }
            resolve(a + b);
        }, 1500);
    });
}

function factorial(num) {
    return new Promise(function (resolve, reject) {
        setTimeout(function () {
            if (num < 0) {
                reject("factorial undefined for negative numbers");
                return;
            }
            let f = 1;
            for (let i = 1; i ≤ num; i++) {
                f *= i;
            }
            resolve(f);
        }, 3000);
    });
}

function square(num) {
    return new Promise(function (resolve, reject) {
        setTimeout(function () {
            resolve(num * num);
```

```
        }, 2000);
    });
}
```

Exam: `new Promise(executor)` — executor gets resolve/reject (names are yours). Resolve data → `.then`; reject data → `.catch`. A function that returns a Promise must be consumed asynchronously.

Pitfall: Treating `let x = sum(2, 3)` as if `x` were already the number `5`. Without `.then` / `await`, `x` is the Promise.

Handling Promises: `.then()` and `.catch()`

Attach handlers on the promise reference

```
somePromise
    .then(function (value) {
        // runs on resolution; value is what resolve( ... ) passed
    })
    .catch(function (err) {
        // runs on rejection; err is what reject( ... ) passed
    });
```

Method	When it runs	Argument
`.then(callback)`	Promise fulfilled	Resolved value
`.catch(callback)`	Promise rejected	Rejected value

One `.catch` at the end of a chain

A **single `.catch()` at the end** handles rejection from **any** earlier step. If step 1 rejects, **all subsequent `.then()` calls are skipped** and control goes straight to `.catch()`.

Chaining `.then()` to avoid callback hell

For dependent operations, **return the next promise from inside `.then()`**, then attach another `.then()`:

```
sum(2, 3)
    .then(function (sumResult) {
        console.log("sum:", sumResult);
        return factorial(sumResult);   // return next promise
    })
    .then(function (factResult) {
        console.log("factorial:", factResult);
        return square(factResult);
    })
    .then(function (sqResult) {
        console.log("square:", sqResult);
    })
    .catch(function (err) {
        console.log("pipeline failed:", err);
    });
```

This stays left-aligned instead of nesting pyramids — the main readability win of Promises over raw callbacks.

Remaining limitation

Chaining still **grows complex** with many operations. The lecture's next tool — **`async` / `await`** — is the cleaner syntax for long sequential pipelines. Promise *definition* does not change; only *handling* syntax changes.

> **Exam:** `.then` on resolve, `.catch` on reject. One trailing `.catch` covers the chain; a rejection skips later `.then`s. Return the next Promise from `.then` to sequence dependent work.

Async / Await

Two ways to handle the same Promises

Style	Best when
`.then()` / `.catch()`	Function must stay visibly asynchronous; no heavy sequencing
`async` / `await`	Multiple async functions must run **sequentially** (dependent ops)

Promise **definition** (`new Promise` , `resolve` , `reject`) does **not** change. Only the consumer syntax changes.

`await`

- Place `await` in front of **any call that returns a Promise**.
- It makes the **rest of the statements inside that function wait** until the promise resolves.
- It **returns the resolved value directly**, so you can assign it to a variable:

```
const sumResult = await sum(2, 3);   // sumResult is a number, not a Promise
```

- **`await` cannot be used outside a function** — it must be inside a function body.
- If you **omit** `await`, the variable receives the **Promise object**, not the resolved value (same trap as Day 4/5 "direct return" confusion).

`async`

- Must be on the **signature of any function that contains** `await`. Without it, JS throws: **`await is only valid in async functions`**.
- Also used as a **declaration** on any function that returns a Promise — signals to other developers "this returns a Promise." In that second use it is largely **declarative / readability**; the important hard rule is: **`await` requires `async`**.

```
async function runPipeline() {
    const sumResult = await sum(2, 3);
    const factResult = await factorial(5);
    const sqResult = await square(12);
    console.log(sumResult, factResult, sqResult);
}

runPipeline();
```

Rejection: `try` / `catch` (no built-in reject handler on `await`)

`await` has **no built-in rejection handler**. **`try` / `catch` fills that role**:

- Put awaited calls inside `try`.
- If **any** promise rejects, control jumps **immediately** to `catch`; **subsequent awaited calls are skipped**.
- `catch` receives the rejected value as its argument.
- More readable than nesting `.then` inside `.catch` for long sequences.

```
async function runPipelineSafe() {
    try {
        const sumResult = await sum(2, 3);
        const factResult = await factorial(sumResult);
```

```
        const sqResult = await square(factResult);
        console.log("done", sumResult, factResult, sqResult);
    } catch (err) {
        console.log("failed:", err);
    }
}

runPipelineSafe();
```

When to use which (lecture rule of thumb)

- **`async` / `await`:** multiple async functions that must run **sequentially** (dependent operations).
- **`.then()` / `.catch()`:** when a function is async and should **remain asynchronous** in style — e.g. fire-and-observe without a long dependent sequence in one function body.

> **Exam:** `await` yields the resolved value and pauses only the async function. Needs `async`. Rejections → `try` / `catch`. Definition of the Promise unchanged.

> **Pitfall:** Writing `const x = sum(2, 3)` inside an `async` function and forgetting `await` — `x` is still a Promise.

DOM: Document Object Model

Problem: the browser does not "paint tags"

When a browser loads an HTML page, it **does not render tags directly** from the file as dumb text. It first **reads all HTML tags and builds an internal tree-like structure**. That tree is the **Document Object Model (DOM)**.

Tree mirrors HTML hierarchy

```
document
└── html
    ├── head
    │   └── title
    │       (link / style if present)
    └── body
        ├── h1
        ├── div
        │   └── p
        └── form
            └── input
```

The browser uses this tree to know **parent / child / sibling** relationships and to render the page. It does **not** re-read the source HTML file for every tiny update; it **references the DOM tree**.

The `document` object

In JavaScript, `document` is the object representing the **entire DOM tree**. **All DOM manipulation** goes through `document` (and element objects obtained from it).

Background: "DOM" is an API standard; browsers expose it to JS so scripts can read and change the live page structure. Changing the DOM updates what the user sees without a full navigation — that is the bridge into the next section.

> **Exam:** DOM = browser's internal tree of the page. `document` = JS handle to that whole tree. Rendering uses the tree, not repeated raw-file rereads.

DOM Manipulation: Core Operations

Definition

DOM manipulation = dynamically modifying the DOM tree **at runtime**, **without reloading the page**. Triggers include user actions (clicks, typing) and other events.

What you can do

Operation	Typical API
Create elements	`document.createElement`
Remove elements	`element.remove()`
Read content	`element.innerHTML` (get)
Change content	`element.innerHTML = " ... "`
Change styles	`element.style.propertyName = " ... "`
Read / set attributes	`getAttribute` / `setAttribute`

Selecting elements

```
const obj = document.getElementById("myId");
```

- Returns the **full element object** (not just its text).
- If nothing matches, you get a nullish miss — the lecture warns that accessing properties on a missing result **throws**. Treat "element not found" as a real failure mode (typo in id, script runs before the element exists, etc.).

Reading and changing content

```
obj.innerHTML;                  // read
obj.innerHTML = "updated content";  // replace content dynamically
```

Works for container-like elements: `h1`, `p`, `div`, `span`, and similar.

Changing styles

CSS property names become **camelCase** in JS:

CSS	JavaScript
`background-color`	`backgroundColor`
`font-size`	`fontSize`
`border-bottom-color`	`borderBottomColor`

```
obj.style.color = "red";
obj.style.backgroundColor = "white";
```

Creating and inserting elements

```
const h = document.createElement("h1");  // exists in memory only
h.innerHTML = "Hello from JS";
const box = document.getElementById("container");
box.appendChild(h);                      // now visible in the tree / page
```

`createElement` alone does **not** show anything. `appendChild` (onto a parent already in the document) inserts it into the live tree.

Removing elements

```
element.remove();   // gone from the DOM tree
```

On **page reload**, the element **reappears** if it still exists in the **source HTML** — removal is a runtime DOM change, not an edit to the file on disk.

Reading and setting attributes — show / hide password

```
element.setAttribute("attrName", "newValue");
element.getAttribute("attrName");
```

Lecture demo pattern: toggle an input between `type="password"` and `type="text"`, and toggle the button label between **Show** and **Hide** using `innerHTML`, with a conditional that checks the button's current `innerHTML` to decide the next state.

```
function togglePassword() {
    const input = document.getElementById("pwd");
    const btn = document.getElementById("toggleBtn");

    if (btn.innerHTML === "Show") {
        input.setAttribute("type", "text");
        btn.innerHTML = "Hide";
    } else {
        input.setAttribute("type", "password");
        btn.innerHTML = "Show";
    }
}
```

Exam: Manipulation changes the live DOM without reload. `getElementById` → element object. `innerHTML` for content; `style.camelCase` for CSS; `createElement` + `appendChild` to insert; `remove()` to delete; `getAttribute` / `setAttribute` for attributes.

Pitfall: Calling `createElement` and wondering why nothing appears — you forgot `appendChild`. Also: assuming `remove()` edits the HTML file permanently — reload restores source markup.

Handling Events in HTML

`onclick` on a button

```
<button onclick="togglePassword()">Show</button>
```

Calls a JS function when the button is clicked.

`onsubmit` on a form — gate before `action`

```
<form action="/login" method="post" onsubmit="return validate()">
```

`onsubmit` return	Effect
`true`	Form **action** URL is allowed to run
`false`	Action is **blocked**

Used to run **validation before submission**. Returning `false` is how you stop a bad form from posting.

`onkeyup` / `onkeydown` on an input

Fires on **every keystroke**. Used for **live validation** as the user types (clear or show error spans immediately).

> **Exam:** `onsubmit` must `return` the validate result; `false` blocks the action.
> `onkeyup` / `onkeydown` power per-keystroke checks.

Form Input, Validation, and Live Error Messages

Reading input values

```
const raw = document.getElementById("fieldId").value;  // always a string
const n = parseInt(raw);                                // numeric extract when needed
```

Trap	Why
`"3" + "5"` → `"35"`	String concatenation, not arithmetic
Forgetting `parseInt`	Math on digit strings silently wrong

Required-field validation with a flag (not early return)

Pattern from the lecture:

1. Start with `isValid = true`.

2. For each field: if empty (`length === 0`), show error in a `<span>` and set `isValid = false` .
3. **Do not return early** on the first failure — early return **stops execution**, so **other fields never get validated**.
4. Return `isValid` at the end.
5. `onsubmit="return validate()"` — if `false` , form action is blocked.

```
function validate() {
    let isValid = true;

    const username = document.getElementById("username").value;
    const password = document.getElementById("password").value;

    if (username.length === 0) {
        document.getElementById("usernameErr").innerHTML = "Username is required";
        isValid = false;
    }

    if (password.length === 0) {
        document.getElementById("passwordErr").innerHTML = "Password is required";
        isValid = false;
    }

    return isValid;
}
```

Live validation on keystroke

Attach `onkeyup` to each input, calling a **field-specific** validate function:

- If field length `> 0` : clear the error span (`innerHTML = ""`).
- If field length `=== 0` : show the error message again.

Username and password were demonstrated; **phone field left as practice**.

```
function validateUsernameLive() {
    const username = document.getElementById("username").value;
    const err = document.getElementById("usernameErr");
    if (username.length > 0) {
        err.innerHTML = "";
    } else {
        err.innerHTML = "Username is required";
    }
}
```

Error message styling

Assign a shared class (e.g. `error-span`) to all error `<span>` elements. One CSS rule covers them all:

```
.error-span {
    color: red;
}
```

Upcoming (next session — not implemented today)

Password format rules were announced for a later class: **8–10 characters**, at least one uppercase and one lowercase, using **regex** for pattern matching. Know that they are coming; today's notes stop at required + live empty checks.

Closing practice tasks from the lecture

- **Bulb on/off toggle:** two images (`bulb-on` , `bulb-off`); toggle `src` on button click — or toggle a div's background between yellow and white.
- Complete **phone** required + live `onkeyup` validation.
- Extend the **portfolio** assignment with concepts so far; finish assignments 2 and 3 if pending.

> **Exam:** `.value` is always a string — `parseInt` for math. Validate with an `isValid` flag so every field is checked. `onsubmit` returns that flag. Live errors: `onkeyup` clears or restores span text. Shared `.error-span { color: red; }` .

> **Pitfall:** `return false` inside the first empty-field `if` — later fields never show errors on the same submit attempt.

Combined Worked Example: Promise Pipeline + Live Form

One HTML file that ties the day's practical threads together: Promise-returning `sum` / `factorial` / `square` , both `.then` chaining and `async` / `await` with `try` / `catch` , plus a small form with required validation, live `onkeyup` errors, and a show/hide password toggle via DOM APIs.

```
<!DOCTYPE html>
<html>
<head>
    <title>WPT Day 5 — Promises, Await, DOM Form</title>
    <style>
```

```
        .error-span {
            color: red;
        }
        body {
            font-family: Georgia, serif;
            max-width: 520px;
            margin: 2rem auto;
            line-height: 1.5;
        }
        label {
            display: block;
            margin-top: 0.75rem;
        }
        #pipelineLog {
            white-space: pre-wrap;
            background: #f4f4f4;
            padding: 0.75rem;
            min-height: 4rem;
        }
    </style>
</head>
<body>
    <h1>Day 5 toolkit</h1>

    <h2>1. Async pipeline</h2>
    <button type="button" onclick="runWithThen()">Run with .then()</button>
    <button type="button" onclick="runWithAwait()">Run with async/await</button>
    <div id="pipelineLog"></div>

    <h2>2. Login-style form</h2>
    <form action="https://example.com/login" method="get"
          onsubmit="return validateForm()">
        <label>
            Username
            <input type="text" id="username" onkeyup="validateUsernameLive()">
        </label>
        <span id="usernameErr" class="error-span"></span>

        <label>
            Password
            <input type="password" id="password" onkeyup="validatePasswordLive()">
        </label>
        <span id="passwordErr" class="error-span"></span>
        <button type="button" id="toggleBtn" onclick="togglePassword()">Show</button>

        <label>
            Phone
            <input type="text" id="phone" onkeyup="validatePhoneLive()">
        </label>
        <span id="phoneErr" class="error-span"></span>
```

```
        <p>
            <button type="submit">Submit</button>
        </p>
    </form>

    <script>
        function logLine(msg) {
            const box = document.getElementById("pipelineLog");
            box.innerHTML = box.innerHTML + msg + "\n";
        }

        function clearLog() {
            document.getElementById("pipelineLog").innerHTML = "";
        }

        // --- Promise-returning helpers (lecture-shaped) ---
        function sum(a, b) {
            return new Promise(function (resolve, reject) {
                setTimeout(function () {
                    if (a > 100 || b > 100) {
                        reject("values must not exceed 100");
                        return;
                    }
                    resolve(a + b);
                }, 400);
            });
        }

        function factorial(num) {
            return new Promise(function (resolve, reject) {
                setTimeout(function () {
                    if (num < 0) {
                        reject("factorial undefined for negative");
                        return;
                    }
                    let f = 1;
                    for (let i = 1; i ≤ num; i++) {
                        f *= i;
                    }
                    resolve(f);
                }, 400);
            });
        }

        function square(num) {
            return new Promise(function (resolve, reject) {
                setTimeout(function () {
                    resolve(num * num);
                }, 400);
            });
        }
```

```
        // --- .then / .catch chain (dependent sequence, flat) ---
        function runWithThen() {
            clearLog();
            logLine("then-chain started…");
            sum(2, 3)
                .then(function (sumResult) {
                    logLine("sum → " + sumResult);
                    return factorial(5);
                })
                .then(function (factResult) {
                    logLine("factorial → " + factResult);
                    return square(12);
                })
                .then(function (sqResult) {
                    logLine("square → " + sqResult);
                    logLine("then-chain done");
                })
                .catch(function (err) {
                    logLine("then-chain failed: " + err);
                });
        }

        // --- async / await + try / catch ---
        async function runWithAwait() {
            clearLog();
            logLine("await pipeline started…");
            try {
                const sumResult = await sum(2, 3);
                logLine("sum → " + sumResult);
                const factResult = await factorial(5);
                logLine("factorial → " + factResult);
                const sqResult = await square(12);
                logLine("square → " + sqResult);
                logLine("await pipeline done");
            } catch (err) {
                logLine("await pipeline failed: " + err);
            }
        }

        // --- Form: flag validation + live errors + attribute toggle ---
        function validateForm() {
            let isValid = true;

            const username = document.getElementById("username").value;
            const password = document.getElementById("password").value;
            const phone = document.getElementById("phone").value;

            if (username.length === 0) {
                document.getElementById("usernameErr").innerHTML = "Username is
required";
```

```
                isValid = false;
            }
            if (password.length === 0) {
                document.getElementById("passwordErr").innerHTML = "Password is
required";
                isValid = false;
            }
            if (phone.length === 0) {
                document.getElementById("phoneErr").innerHTML = "Phone is required";
                isValid = false;
            }

            return isValid;
        }

        function validateUsernameLive() {
            const v = document.getElementById("username").value;
            document.getElementById("usernameErr").innerHTML =
                v.length > 0 ? "" : "Username is required";
        }

        function validatePasswordLive() {
            const v = document.getElementById("password").value;
            document.getElementById("passwordErr").innerHTML =
                v.length > 0 ? "" : "Password is required";
        }

        function validatePhoneLive() {
            const v = document.getElementById("phone").value;
            document.getElementById("phoneErr").innerHTML =
                v.length > 0 ? "" : "Phone is required";
        }

        function togglePassword() {
            const input = document.getElementById("password");
            const btn = document.getElementById("toggleBtn");
            if (btn.innerHTML === "Show") {
                input.setAttribute("type", "text");
                btn.innerHTML = "Hide";
            } else {
                input.setAttribute("type", "password");
                btn.innerHTML = "Show";
            }
        }
    </script>
</body>
</html>
```

Trace it end to end

1. **Promise helpers.** `sum` / `factorial` / `square` each return a `new Promise`; the executor schedules work with `setTimeout` and later `resolve`s or `reject`s — same contract as the lecture.
2. `.then` **path.** Each handler logs, then **returns** the next Promise so the chain stays sequential without callback pyramids. One trailing `.catch` covers any step.
3. `async` / `await` **path.** Same sequence, values assigned directly (`const sumResult = await sum( … )`). Failures jump to `catch`; later awaits are skipped.
4. **DOM log.** `logLine` uses `innerHTML` on `#pipelineLog` — runtime content change, no reload.
5. **Submit gate.** `onsubmit="return validateForm()"` — `isValid` flag checks **all** fields before returning; empty fields get red `.error-span` messages.
6. **Live errors.** Each `onkeyup` clears or restores that field's span as the user types.
7. **Show/Hide.** `setAttribute("type", … )` plus button `innerHTML` toggle — attribute + content manipulation together.

If you can explain those seven steps without scrolling up, Day 5's async-to-DOM arc is under control.

Quick Revision Checklist

Topic	One-line takeaway
Callback timing	Never immediate — waits for timer / server / DB / event
Library async	Stays async; put dependent work in the completion path
DB then respond	`sendResponse` inside read's callback, not on the next line
Dependent nest	Output of N feeds N+1 → nest (or Promise/ `await` chain)
Independent work	Run in parallel; nesting only wastes time
Nested timers demo	Sum 1.5s → fact 3s → square 2s ≈ **6.5s** total
Callback hell	Pyramid nesting → hard to read, debug, maintain, scale
Promise states	Pending → Fulfilled or Rejected
Forgotten settle	Never resolve/reject → pending forever

Topic	One-line takeaway
Executor	Runs immediately when `new Promise` is created
resolve / reject	Not keywords; any names; data → `.then` / `.catch`
Return value	Promise-returning fn → consume with `.then` / `await`, not sync assign
`.then` chain	Return next Promise; one end `.catch` skips later thens on reject
`await`	Pauses async function; yields resolved value to a variable
`async`	Required on any function that uses `await`
Await rejection	Use `try` / `catch`; later awaits skipped after reject
When which	`async` / `await` for dependent sequences; `.then` when staying async-styled
DOM	Browser's HTML tree; `document` is the JS handle
Manipulation	Change tree at runtime without reload
`getElementById`	Full element object; missing id → dangerous property access
`innerHTML`	Read/set element content
`style`	camelCase CSS names (`backgroundColor`)
Create/insert	`createElement` then `appendChild`
`remove()`	Drops from live tree; reload restores source HTML
Attributes	`getAttribute` / `setAttribute` (e.g. password show/hide)
`onsubmit`	Return `true` / `false` to allow/block action
`.value`	Always string; `parseInt` for arithmetic
Validate flag	Check all fields; avoid early `return`
Live validate	`onkeyup` clears or shows error spans
Error CSS	Shared class e.g. `.error-span { color: red; }`

Practice Exercises

Concept

1. Why must `sendResponse` be placed inside `readDataFromDbAsync`'s callback rather than on the following line?
2. Name the three Promise states and what "never calling resolve or reject" causes.
3. What is the DOM, and what JS object represents the whole tree?
4. Why does the lecture prefer an `isValid` flag over returning `false` at the first empty field?

Application

5. Rewrite a three-level nested `setTimeout` chain (sum → factorial → square) as a `.then` chain using Promise-returning helpers.
6. Write an `async` function that `await`s the same three helpers inside `try` / `catch`.
7. Implement show/hide password with `getElementById`, `getAttribute` / `setAttribute`, and button `innerHTML` toggling.
8. Add phone required + live `onkeyup` validation consistent with username/password.

Predict output

9. Independent timers: sum delay 1.5s, factorial 3s, square 2s — in what order do the **results** log, and roughly when does the last one finish?
10. Dependent nested timers with the same delays — what is the approximate total wait, and why?
11. Inside `async function f() { const x = sum(2, 3); console.log(x); }` with no `await` — what kind of value is logged?

Spot the bug

12. Find the bug:

```
readDataFromDbAsync(function (rows) {
    console.log("got rows", rows);
});
sendResponse(rows);
```

13. Find the bug:

```
function validate() {
    if (username.length == 0) {
        usernameErr.innerHTML = "required";
        return false;
    }
```

```
    if (password.length === 0) {
        passwordErr.innerHTML = "required";
        return false;
    }
    return true;
}
```

14. Find the bug:

```
const el = document.createElement("p");
el.innerHTML = "Hi";
// user expects to see "Hi" on the page immediately
```

15. Find the bug:

```
<form action="/save" onsubmit="validate()">
```

(Assume `validate` returns `false` on error.)

Practice Answers

1. Because the DB read is async: the callback runs only when data arrives. The next line runs immediately, so `sendResponse` would fire before rows exist. Dependent use of the rows belongs inside the callback.

2. Pending, Fulfilled (Resolved), Rejected. Never settling leaves the Promise pending forever.

3. The DOM is the browser's internal tree built from HTML tags. The `document` object in JavaScript represents that entire tree.

4. An early `return false` stops the function so later fields are never checked or given error messages. A flag lets every field run, then returns the combined result.

5.

```
sum(2, 3)
    .then(function (s) { return factorial(5); })
    .then(function (f) { return square(12); })
    .then(function (q) { console.log(q); })
    .catch(function (e) { console.log(e); });
```

6.

```
async function run() {
    try {
        const s = await sum(2, 3);
        const f = await factorial(5);
        const q = await square(12);
        console.log(s, f, q);
    } catch (e) {
        console.log(e);
    }
}
```

7.

```
function togglePassword() {
    const input = document.getElementById("password");
    const btn = document.getElementById("toggleBtn");
    if (btn.innerHTML === "Show") {
        input.setAttribute("type", "text");
        btn.innerHTML = "Hide";
    } else {
        input.setAttribute("type", "password");
        btn.innerHTML = "Show";
    }
}
```

8.

```
function validatePhoneLive() {
    const v = document.getElementById("phone").value;
    document.getElementById("phoneErr").innerHTML =
        v.length > 0 ? "" : "Phone is required";
}
```

Also include `phone.length === 0` in the submit-time `validate` flag checks.

9. Results log roughly: sum (1.5s), then square (2s), then factorial (3s). Last finishes at about **3s** (longest delay), because independent timers overlap.

10. About **6.5s** (1.5 + 3 + 2). Nesting starts each timer only after the previous callback, so delays add.

11. A **Promise** object (not the number `5`). Without `await`, assignment captures the Promise itself.

12. `rows` is only in scope inside the callback; `sendResponse(rows)` outside both runs too early and references an outer `rows` that is not the DB result. Move `sendResponse(rows)` inside the

callback.

13. Early `return false` on username means an empty password never gets its error span updated on that submit. Use `isValid = false` and return once at the end.

14. `createElement` only builds a node in memory. Without `parent.appendChild(el)`, it never enters the live DOM, so nothing appears.

15. Missing `return` in `onsubmit="validate()"`. Even if `validate` returns `false`, the handler's return value is ignored unless written `onsubmit="return validate()"`, so the form may still submit.